Feature Checklist Report

Feature Checklist Implementation Report
Repository: react_test
Generated from the current workspace source files

This report maps each checklist requirement to the files and exact line ranges that implement it. The excerpts are read directly from the repository during generation.

==========

MP1 - Product CRUD, multiple photos, and bulk checkbox actions
Admins can create, view, update, and delete products. Product creation and updates accept multiple images, while the admin product table tracks selected IDs for bulk actions.

backend/routes/product.js (lines 24-32)

```
  24: router.get('/product/:id', getSingleProduct)
  25: router.get('/products', getProducts)
  26: router.put('/admin/product/:id', isAuthenticatedUser, authorizeRoles('admin'), upload.array('images', 10), up
      dateProduct);
  27: router.delete('/admin/product/:id', isAuthenticatedUser, authorizeRoles('admin'), deleteProduct);
  28: router.delete('/admin/products/bulk', isAuthenticatedUser, authorizeRoles('admin'), bulkDeleteProducts);
  29: router.patch('/admin/products/bulk/archive', isAuthenticatedUser, authorizeRoles('admin'), bulkArchiveProduct
      s);
  30: router.patch('/admin/products/bulk/unarchive', isAuthenticatedUser, authorizeRoles('admin'), bulkUnarchivePro
      ducts);
  31: router.get('/admin/products', isAuthenticatedUser, authorizeRoles('admin'), getAdminProducts);
  32: router.post('/admin/product/new', isAuthenticatedUser, upload.array('images', 10), newProduct);
```

backend/controllers/product.js (lines 25-82)

```
  25: exports.newProduct = async (req, res, next) => {
  26:     let images = []
  27:     // if (typeof req.files === 'string') {
  28:     //     images.push(req.files)
  29:     // } else {
  30:     //     images = req.files
  31:     // }
  32:
  33:     if (typeof req.body.images === 'string') {
  34:         images.push(req.body.images)
  35:     } else {
  36:         images = req.body.images || []
  37:     }
  38:
  39:     let imagesLinks = [];
  40:
  41:     for (let i = 0; i < images.length; i++) {
  42:         try {
  43:             // const result = await cloudinary.v2.uploader.upload(images[i].path, {
  44:             //     folder: 'products',
  45:             //     width: 150,
  46:             //     crop: "scale",
  47:             // });
  48:
  49:             const result = await cloudinary.v2.uploader.upload(images[i], {
  50:                 folder: 'products',
```

Feature Checklist Report

```
                width: 150,
                crop: "scale",
            });

            imagesLinks.push({
                public_id: result.public_id,
                url: result.secure_url
            })

        } catch (error) {
            console.log(error)
        }

    }

    const category = await resolveCategoryId(req.body.category_id, req.body.category)
    if (!category) {
        return res.status(400).json({ success: false, message: 'Please select a valid category' })
    }

    const product = await Product.create({
        ...req.body,
        images: imagesLinks,
        category_id: category._id,
        category: undefined,
        user: undefined
    });

    if (!product)
        return res.status(400).json({
            success: false,
            message: 'Product not created'
```

backend/controllers/product.js (lines 166-218)

```
exports.updateProduct = async (req, res, next) => {
    try {
        let product = await Product.findById(req.params.id);
        if (!product) {
            return res.status(404).json({
                success: false,
                message: `Product with ID ${req.params.id} was not found`
            })
        }

        const images = req.body.images
            ? (Array.isArray(req.body.images) ? req.body.images : [req.body.images])
            : []
        let imagesLinks = product.images

        if (images.length > 0) {
            imagesLinks = []
            for (const image of images) {
                try {
```

Feature Checklist Report

```
                    const result = await cloudinary.v2.uploader.upload(image, {
                        folder: 'products',
                        width: 150,
                        crop: "scale",
                    })
                    imagesLinks.push({
                        public_id: result.public_id,
                        url: result.secure_url
                    })
                } catch (error) {
                    return res.status(502).json({
                        success: false,
                        message: 'Product image upload failed: ${error.message}'
                    })
                }
            }
        }

        const category = await resolveCategoryId(req.body.category_id, req.body.category)
        if (!category) {
            return res.status(400).json({
                success: false,
                message: 'Product update failed: provide a valid category name or category ID'
            })
        }

        product = await Product.findByIdAndUpdate(req.params.id, {
            ...req.body,
            images: imagesLinks,
            category_id: category._id,
            category: undefined,
            user: undefined
        }, {
            new: true,
```

backend/controllers/product.js (lines 231-310)

```
exports.deleteProduct = async (req, res, next) => {
    const product = await Product.findByIdAndDelete(req.params.id);
    if (!product) {
        return res.status(404).json({
            success: false,
            message: 'Product not found'
        })
    }

    return res.status(200).json({
        success: true,
        message: 'Product deleted'
    })
}

function getProductIds(req, res) {
    const { ids } = req.body || {}
```

Feature Checklist Report

```
    if (!Array.isArray(ids) || ids.length === 0 || ids.some(id => !mongoose.isValidObjectId(id))) {
        res.status(400).json({
            success: false,
            message: 'Please provide a non-empty array of valid product IDs'
        })
        return null
    }
    return [...new Set(ids.map(id => id.toString()))]
}

exports.bulkDeleteProducts = async (req, res, next) => {
    try {
        const ids = getProductIds(req, res)
        if (!ids) return

        await Promise.all([
            Review.deleteMany({ product_id: { $in: ids } }),
            Blueprint.updateMany(
                { extracted_materials: { $in: ids } },
                { $pull: { extracted_materials: { $in: ids } } }
            )
        ])
        const result = await Product.deleteMany({ _id: { $in: ids } })

        return res.status(200).json({
            success: true,
            deletedCount: result.deletedCount
        })
    } catch (error) {
        return next(error)
    }
}

exports.bulkArchiveProducts = async (req, res, next) => {
    try {
        const ids = getProductIds(req, res)
        if (!ids) return

        const result = await Product.updateMany(
            { _id: { $in: ids }, isArchived: { $ne: true } },
            { $set: { isArchived: true } }
        )

        return res.status(200).json({
            success: true,
            archivedCount: result.modifiedCount
        })
    } catch (error) {
        return next(error)
    }
}

exports.bulkUnarchiveProducts = async (req, res, next) => {
```

Feature Checklist Report

```
    try {
        const ids = getProductIds(req, res)
        if (!ids) return

        const result = await Product.updateMany(
            { _id: { $in: ids }, isArchived: true },
            { $set: { isArchived: false } }
        )

        return res.status(200).json({
```

frontend/src/actions/productActions.jsx (lines 107-223)

```
export const newProduct = (productData) => async (dispatch) => {
    try {
        dispatch({ type: NEW_PRODUCT_REQUEST })
        const config = {
            headers: {
                'Content-Type': 'application/json'
            },
            withCredentials: true //correct
        }
        const { data } = await axios.post(`${import.meta.env.VITE_API}/admin/product/new`, productData, confi
    g)
        dispatch({
            type: NEW_PRODUCT_SUCCESS,
            payload: data
        })
    } catch (error) {
        dispatch({
            type: NEW_PRODUCT_FAIL,
            payload: error.response.data.message
        })
    }
}

export const getAdminProducts = () => async (dispatch) => {
    try {
        dispatch({ type: ADMIN_PRODUCTS_REQUEST })
        const { data } = await axios.get(`${import.meta.env.VITE_API}/admin/products`, {
            //AxiosRequestConfig parameter
            withCredentials: true //correct
        })
        dispatch({
            type: ADMIN_PRODUCTS_SUCCESS,
            payload: data.products
        })
    } catch (error) {
        dispatch({
            type: ADMIN_PRODUCTS_FAIL,
            payload: error.response.data.message
        })
    }
}
```

Feature Checklist Report

```
147:
148: export const updateProduct = (id, productData) => async (dispatch) => {
149:     try {
150:         dispatch({ type: UPDATE_PRODUCT_REQUEST })
151:         const config = {
152:             withCredentials: true //correct
153:         }
154:         const { data } = await axios.put(`${import.meta.env.VITE_API}/admin/product/${id}`, productData, conf
     ig)
155:         dispatch({
156:             type: UPDATE_PRODUCT_SUCCESS,
157:             payload: data
158:         })
159:     } catch (error) {
160:         dispatch({
161:             type: UPDATE_PRODUCT_FAIL,
162:             payload: error.response?.data?.message || error.message || 'Unable to update product'
163:         })
164:     }
165: }
166:
167: export const deleteProduct = (id) => async (dispatch) => {
168:     try {
169:
170:         dispatch({ type: DELETE_PRODUCT_REQUEST })
171:
172:         const { data } = await axios.delete(`${import.meta.env.VITE_API}/admin/product/${id}`, {
173:             withCredentials: true
174:         })
175:
176:         dispatch({
177:             type: DELETE_PRODUCT_SUCCESS,
178:             payload: data.success
179:         })
180:
181:     } catch (error) {
182:         dispatch({
183:             type: DELETE_PRODUCT_FAIL,
184:             payload: error.response?.data?.message || error.message || 'Unable to delete product'
185:         })
186:     }
187: }
188:
189: export const bulkDeleteProducts = (ids) => async (dispatch) => {
190:     try {
191:         dispatch({ type: BULK_DELETE_PRODUCTS_REQUEST })
192:         const { data } = await axios.delete(`${import.meta.env.VITE_API}/admin/products/bulk`, {
193:             data: { ids },
194:             withCredentials: true
195:         })
196:         dispatch({ type: BULK_DELETE_PRODUCTS_SUCCESS, payload: data.deletedCount })
197:     } catch (error) {
198:         dispatch({
```

Feature Checklist Report

```
            type: BULK_DELETE_PRODUCTS_FAIL,
            payload: error.response?.data?.message || error.message || 'Unable to delete products'
        })
    }
}

export const bulkArchiveProducts = (ids) => async (dispatch) => {
    try {
        dispatch({ type: BULK_ARCHIVE_PRODUCTS_REQUEST })
        const { data } = await axios.patch(`${import.meta.env.VITE_API}/admin/products/bulk/archive`, { ids }, {
            withCredentials: true
        })
        dispatch({ type: BULK_ARCHIVE_PRODUCTS_SUCCESS, payload: data.archivedCount })
    } catch (error) {
        dispatch({
            type: BULK_ARCHIVE_PRODUCTS_FAIL,
            payload: error.response?.data?.message || error.message || 'Unable to archive products'
        })
    }
}

export const bulkUnarchiveProducts = (ids) => async (dispatch) => {
    try {
        dispatch({ type: BULK_UNARCHIVE_PRODUCTS_REQUEST })
        const { data } = await axios.patch(`${import.meta.env.VITE_API}/admin/products/bulk/unarchive`, { ids }, {
```

frontend/src/Components/Admin/NewProduct.jsx (lines 51-82)

```
    const formik = useFormik({
        initialValues: { name: '', price: 0, description: '', category: 'Materials', stock: 0, seller: '' },
        validationSchema: Yup.object({
            name: Yup.string().trim().required('Product name is required'),
            price: Yup.number().min(0, 'Price cannot be negative').required('Price is required'),
            description: Yup.string().trim().required('Description is required'),
            category: Yup.string().required('Category is required'),
            stock: Yup.number().integer().min(0, 'Stock cannot be negative').required('Stock is required'),
            seller: Yup.string().trim().required('Seller name is required')
        }),
        onSubmit: values => submitHandler(values)
    })

    const submitHandler = (values) => {
        if (imageError) {
            toast.error(imageError, { position: 'bottom-right' })
            return
        }

        const formData = new FormData();
        formData.set('name', values.name);
        formData.set('price', values.price);
        formData.set('description', values.description);
        formData.set('category', values.category);
```

Feature Checklist Report

```
        formData.set('stock', values.stock);
        formData.set('seller', values.seller);

        images.forEach(image => {
            formData.append('images', image)
        })

        dispatch(newProduct(formData))
```

frontend/src/Components/Admin/ProductsList.jsx (lines 27-88)

```
import MetaData from '../Layout/MetaData'
import Sidebar from './SideBar'
import {
    getAdminProducts,
    bulkDeleteProducts,
    bulkArchiveProducts,
    bulkUnarchiveProducts,
    clearErrors,
} from '../../actions/productActions'
import { BULK_PRODUCTS_RESET } from '../../constants/productConstants'

const ProductsList = () => {
    const dispatch = useDispatch()
    const { loading, error, products = [] } = useSelector(state => state.products)
    const { error: deleteError, isBulkActionComplete, affectedCount, loading: actionLoading } = useSelector(state => state.product)
    const [selectedIds, setSelectedIds] = useState([])
    const [confirmation, setConfirmation] = useState(null)
    const [searchTerm, setSearchTerm] = useState('')

    const selectedProducts = products.filter(product => selectedIds.includes(product._id))
    const allSelectedProductsArchived = selectedProducts.length > 0 && selectedProducts.every(product => product.isArchived)

    useEffect(() => {
        if (error) {
            toast.error(error, { position: 'bottom-right' })
            dispatch(clearErrors())
        }

        if (deleteError) {
            toast.error(deleteError, { position: 'bottom-right' })
            dispatch(clearErrors())
        }

        if (isBulkActionComplete) {
            toast.success(`${affectedCount} product${affectedCount === 1 ? '' : 's'} processed successfully`, {
                position: 'bottom-right'
            })
            dispatch(getAdminProducts())
            dispatch({ type: BULK_PRODUCTS_RESET })
        }
```

Feature Checklist Report

```
    }, [dispatch, error, deleteError, isBulkActionComplete, affectedCount])

    useEffect(() => {
        dispatch(getAdminProducts())
    }, [dispatch])

    const bulkActionHandler = (action, ids = selectedIds, productName = '') => {
        setConfirmation({ action, ids, productName })
    }

    const confirmAction = () => {
        if (!confirmation) return
        const { action, ids } = confirmation
        setConfirmation(null)
        setSelectedIds([])
        dispatch(
            action === 'delete'
                ? bulkDeleteProducts(ids)
                : action === 'archive'
                    ? bulkArchiveProducts(ids)
                    : bulkUnarchiveProducts(ids)
        )
```

==========

MP2 - Product filter functions

The frontend serializes price, category, rating, review-count, keyword, and page filters into the products request. The backend translates them into MongoDB conditions and pagination.

frontend/src/actions/productActions.jsx (lines 45-80)

```
export const getProducts = (
    currentPage = 1,
    keyword = '',
    price = [0, 100000],
    category = '',
    rating = '',
    reviews = '',
    append = false
) => async (dispatch) => {
    try {
        dispatch({
            type: ALL_PRODUCTS_REQUEST,
            append
        })

        const params = new URLSearchParams({
            page: currentPage.toString(),
            limit: '8',
            keyword,
            'price[lte]': price[1].toString(),
            'price[gte]': price[0].toString()
        })
        if (Array.isArray(category)) {
            category.forEach(categoryId => params.append('category', categoryId))
```

Feature Checklist Report

```
        } else if (category) {
            params.set('category', category)
        }
        if (rating) params.set('rating[gte]', rating.toString())
        if (reviews) params.set('numOfReviews[gte]', reviews.toString())
        const link = `${import.meta.env.VITE_API}/products?${params.toString()}`
        const { data } = await axios.get(link)
        console.log(data)
        dispatch({
            type: ALL_PRODUCTS_SUCCESS,
            payload: data,
            meta: { append }
```

frontend/src/Components/Home.jsx (lines 151-185)

```
        return () => {
            isMounted = false
        }
    }, [])

    useEffect(() => {
        if (queryCategory) {
            if (queryCategory === 'all') {
                setCategory([])
            } else if (categories.length > 0) {
                const matched = categories.find(
                    c => c._id === queryCategory || c.name.toLowerCase() === queryCategory.toLowerCase()
                )
                setCategory(matched ? [matched._id] : [queryCategory])
            } else {
                setCategory([queryCategory])
            }
            setCurrentPage(1)
        }
    }, [queryCategory, categories])

    useEffect(() => {
        dispatch(getProducts(currentPage, keyword, price, category, rating, reviews, currentPage > 1))
    }, [currentPage, keyword, price, category, rating, reviews, dispatch]);

    useEffect(() => {
        const loadMore = entries => {
            if (entries[0].isIntersecting && !loading && products.length < filteredProductsCount) {
                setCurrentPage(page => page + 1)
            }
        }
        const observer = new IntersectionObserver(loadMore, { rootMargin: '300px' })
        if (loadMoreRef.current) observer.observe(loadMoreRef.current)
        return () => observer.disconnect()
    }, [filteredProductsCount, loading, products.length])
```

backend/utils/apiFeatures.js (lines 14-91)

```
                $options: 'i'
            }
```

Feature Checklist Report

```
        } : {}
        // console.log(this.query, this.queryStr);
        this.query = this.query.find({ ...keyword });
        return this;
    }

    filter() {
        // { 'price[gte]': '100', 'price[lte]': '1000' }
        const queryCopy = { ...this.queryStr };
        // console.log(queryCopy);
        // Removing fields from the query
        const removeFields = ['keyword', 'page', 'limit']
        removeFields.forEach(el => delete queryCopy[el]);

        let priceFilter = {};
        const priceGte = queryCopy['price[gte]'] || queryCopy.price?.gte;
        const priceLte = queryCopy['price[lte]'] || queryCopy.price?.lte;
        if (priceGte || priceLte) {
            priceFilter.price = {};
            if (priceGte) {
                priceFilter.price.$gte = Number(priceGte);
            }
            if (priceLte) {
                priceFilter.price.$lte = Number(priceLte);
            }
            // priceFilter {
            //     price: {
            //         $gte: 100,
            //         $lte: 1000
            //     }
            // }
            delete queryCopy['price[gte]'];
            delete queryCopy['price[lte]'];
            delete queryCopy.price;
        }

        if (queryCopy.category) {
            queryCopy.category_id = Array.isArray(queryCopy.category)
                ? { $in: queryCopy.category }
                : queryCopy.category;
            delete queryCopy.category;
        }

        const ratingGte = queryCopy['rating[gte]'] || queryCopy.rating?.gte;
        if (ratingGte) {
            queryCopy.ratings = { $gte: Number(ratingGte) };
            delete queryCopy['rating[gte]'];
            delete queryCopy.rating;
        }

        const reviewsGte = queryCopy['numOfReviews[gte]'] || queryCopy.numOfReviews?.gte;
        if (reviewsGte) {
            delete queryCopy['numOfReviews[gte]'];
```

Feature Checklist Report

```
            delete queryCopy.numOfReviews;
            queryCopy.numOfReviews = { $gte: Number(reviewsGte) };
        }

        delete queryCopy['price[gte]'];
        delete queryCopy['price[lte]'];
        delete queryCopy.isArchived;

        // console.log(queryCopy);

        this.query = this.query.find({ ...priceFilter, ...queryCopy });
        return this;
    }

    pagination(resPerPage) {
        const currentPage = Number(this.queryStr.page) || 1;
        const skip = resPerPage * (currentPage - 1);

        this.query = this.query.limit(resPerPage).skip(skip);
        return this;
    }
}
module.exports = APIFeatures
```

backend/controllers/product.js (lines 107-145)

```
        name: review.user_id?.name,
        avatar: review.user_id?.avatar
    }))
    return res.status(200).json({
        success: true,
        product: productResponse
    })
}

exports.getProducts = async (req, res) => {

    const resPerPage = Number(req.query.limit) || 8;
    const productsCount = await Product.countDocuments({ isArchived: { $ne: true } });

   // const products = await Product.find()
    const apiFeatures = new APIFeatures(Product.find({ isArchived: { $ne: true } }), req.query).search().filter()

    const filteredProductsCount = await Product.countDocuments(apiFeatures.query.getFilter());

    apiFeatures.query = apiFeatures.query.sort({ createdAt: -1 });
    apiFeatures.pagination(resPerPage);
    const products = await apiFeatures.query.populate('category_id');

    if (!products)
        return res.status(400).json({ message: 'error loading products' })
    return res.status(200).json({
        success: true,
```

Feature Checklist Report

```
134:         products,
135:         filteredProductsCount,
136:         resPerPage,
137:         productsCount,
138:
139:     })
140:     // if (!products)
141:     //     return res.status(400).json({ message: 'error loading products' })
142:
143:     // return res.status(200).json({
144:     //     success: true,
145:     //     products,
```

==========

MP3 - Reviews and ratings

A logged-in customer can submit or update one review after receiving the product. The backend uses an upsert, recalculates the product average/count, and restricts deletion to admins.

frontend/src/Components/Product/ProductDetails.jsx (lines 25-45)

```
25:     const decreaseQty = () => setQuantity(current => Math.max(current - 1, 1))
26:     const addToCart = () => dispatch(addItemToCart(id, quantity))
27:
28:     const openReviewDialog = () => {
29:         setRating(review?.rating || 0)
30:         setComment(review?.comment || '')
31:         setReviewDialogOpen(true)
32:     }
33:
34:     const reviewHandler = async () => {
35:         if (rating < 1 || !comment.trim()) return
36:         const saved = await dispatch(newReview({ rating, comment: comment.trim(), productId: id }))
37:         if (!saved) return
38:         setReviewDialogOpen(false)
39:         dispatch(getProductDetails(id))
40:         dispatch(getMyProductReview(id))
41:     }
42:
43:     useEffect(() => {
44:         dispatch(getProductDetails(id))
45:     }, [dispatch, id])
```

frontend/src/actions/productActions.jsx (lines 235-305)

```
235: export const newReview = (reviewData) => async (dispatch) => {
236:     try {
237:         dispatch({ type: NEW_REVIEW_REQUEST })
238:         const { data } = await axios.put(`${import.meta.env.VITE_API}/review`, reviewData, {
239:             headers: { 'Content-Type': 'application/json' },
240:             withCredentials: true
241:         })
242:         dispatch({
243:             type: NEW_REVIEW_SUCCESS,
244:             payload: data
245:         })
```

Feature Checklist Report

```
        return true
    } catch (error) {
        dispatch({
            type: NEW_REVIEW_FAIL,
            payload: error.response?.data?.message || error.message || 'Unable to save review'
        })
        return false
    }
}

export const getMyProductReview = (productId) => async (dispatch) => {
    try {
        dispatch({ type: MY_REVIEW_REQUEST })
        const { data } = await axios.get(`${import.meta.env.VITE_API}/review?productId=${productId}`, {
            withCredentials: true
        })
        dispatch({ type: MY_REVIEW_SUCCESS, payload: data })
    } catch (error) {
        dispatch({
            type: MY_REVIEW_FAIL,
            payload: error.response?.data?.message || error.message || 'Unable to load review status'
        })
    }
}

export const getProductReviews = (productId) => async (dispatch) => {
    try {
        dispatch({ type: GET_REVIEWS_REQUEST })
        const { data } = await axios.get(`${import.meta.env.VITE_API}/reviews?id=${encodeURIComponent(product
    Id)}`, {
            withCredentials: true
        })
        dispatch({ type: GET_REVIEWS_SUCCESS, payload: data.reviews })
    } catch (error) {
        dispatch({
            type: GET_REVIEWS_FAIL,
            payload: error.response?.data?.message || error.message || 'Unable to load reviews'
        })
    }
}

export const deleteReview = (reviewId, productId) => async (dispatch) => {
    try {
        dispatch({ type: DELETE_REVIEW_REQUEST })
        await axios.delete(
            `${import.meta.env.VITE_API}/reviews?id=${encodeURIComponent(reviewId)}&productId=${encodeURIComp
    onent(productId)}`,
            { withCredentials: true }
        )
        dispatch({ type: DELETE_REVIEW_SUCCESS, payload: reviewId })
        return true
    } catch (error) {
        dispatch({
```

Feature Checklist Report

```
297:             type: DELETE_REVIEW_FAIL,
298:             payload: error.response?.data?.message || error.message || 'Unable to delete review'
299:         })
300:         return false
301:     }
302: }
303:
304: export const clearErrors = () => async (dispatch) => {
305:     dispatch({
```

backend/routes/product.js (lines 33-36)

```
 33: router.put('/review', isAuthenticatedUser, createProductReview);
 34: router.get('/review', isAuthenticatedUser, getMyProductReview);
 35: router.get('/reviews', isAuthenticatedUser, getProductReviews)
 36: router.delete('/reviews', isAuthenticatedUser, authorizeRoles('admin'), deleteReview)
```

backend/controllers/product.js (lines 319-423)

```
319: exports.createProductReview = async (req, res, next) => {
320:     const { rating, comment, productId } = req.body;
321:     const numericRating = Number(rating)
322:     const filteredComment = filterComment(comment)
323:     if (!Number.isInteger(numericRating) || numericRating < 1 || numericRating > 5 || !filteredComment) {
324:         return res.status(400).json({ success: false, message: 'Please provide a rating from 1 to 5 and a rev
    iew' })
325:     }
326:     const product = await Product.findById(productId);
327:     if (!product) {
328:         return res.status(404).json({ success: false, message: 'Product not found' })
329:     }
330:
331:     const hasPurchased = await Order.exists({
332:         user: req.user._id,
333:         orderStatus: 'Delivered',
334:         'orderItems.product': product._id
335:     })
336:     if (!hasPurchased) {
337:         return res.status(403).json({
338:             success: false,
339:             message: 'You can review a product only after purchasing and receiving it'
340:         })
341:     }
342:
343:     await Review.findOneAndUpdate(
344:         { user_id: req.user._id, product_id: productId },
345:         { rating: numericRating, comment: filteredComment },
346:         { new: true, upsert: true, runValidators: true, setDefaultsOnInsert: true }
347:     )
348:     const stats = await Review.aggregate([
349:         { $match: { product_id: product._id } },
350:         { $group: { _id: null, average: { $avg: '$rating' }, count: { $sum: 1 } } }
351:     ])
352:     await Product.findByIdAndUpdate(productId, {
353:         ratings: stats[0]?.average || 0,
```

Feature Checklist Report

```
            numOfReviews: stats[0]?.count || 0
        })
        return res.status(200).json({
            success: true,
            message: 'Review saved successfully'
        })
}

exports.getMyProductReview = async (req, res, next) => {
    try {
        const product = await Product.findById(req.query.productId).select('_id')
        if (!product) {
            return res.status(404).json({ success: false, message: 'Product not found' })
        }

        const hasPurchased = await Order.exists({
            user: req.user._id,
            orderStatus: 'Delivered',
            'orderItems.product': product._id
        })
        if (!hasPurchased) {
            return res.status(200).json({ success: true, canReview: false, review: null })
        }

        const review = await Review.findOne({
            user_id: req.user._id,
            product_id: product._id
        })
        return res.status(200).json({
            success: true,
            canReview: true,
            review: review
                ? { ...review.toObject(), comment: filterComment(review.comment) }
                : null
        })
    } catch (error) {
        return next(error)
    }
}

exports.getProductReviews = async (req, res, next) => {
    const reviews = await Review.find({ product_id: req.query.id })
        .populate('user_id', 'name avatar')
        .sort({ createdAt: -1 })
    res.status(200).json({
        success: true,
        reviews: reviews.map(review => ({
            ...review.toObject(),
            comment: filterComment(review.comment),
            user: review.user_id?._id,
            name: review.user_id?.name,
            avatar: review.user_id?.avatar
        }))
```

Feature Checklist Report

```
    })
}

exports.deleteReview = async (req, res, next) => {
    await Review.findByIdAndDelete(req.query.id)
    const stats = await Review.aggregate([
        { $match: { product_id: new mongoose.Types.ObjectId(req.query.productId) } },
        { $group: { _id: null, average: { $avg: '$rating' }, count: { $sum: 1 } } }
    ])
    await Product.findByIdAndUpdate(req.query.productId, {
        ratings: stats[0]?.average || 0,
        numOfReviews: stats[0]?.count || 0
    })

    return res.status(200).json({
        success: true
    })
```

==========

Term test lab - Completed transactions, status updates, email, and PDF receipt
Orders reserve stock and are saved as completed transactions. Admin status changes are persisted, then the customer receives an email containing order items, subtotal, shipping, tax, grand total, and a generated PDF receipt attachment.

frontend/src/actions/orderActions.jsx (lines 1-45)

```
import axios from 'axios'
import {
    CREATE_ORDER_REQUEST,
    CREATE_ORDER_SUCCESS,
    CREATE_ORDER_FAIL,
    MY_ORDERS_REQUEST,
    MY_ORDERS_SUCCESS,
    MY_ORDERS_FAIL,
    ORDER_DETAILS_REQUEST,

    ORDER_DETAILS_SUCCESS,

    ORDER_DETAILS_FAIL,
    ALL_ORDERS_REQUEST,

    ALL_ORDERS_SUCCESS,

    ALL_ORDERS_FAIL,
    UPDATE_ORDER_SUCCESS,

    UPDATE_ORDER_REQUEST,

    UPDATE_ORDER_FAIL,
    DELETE_ORDER_REQUEST,
    DELETE_ORDER_SUCCESS,
    DELETE_ORDER_FAIL,
    CLEAR_ERRORS

```

Feature Checklist Report

```
} from '../constants/orderConstants'

export const createOrder = (order) => async (dispatch) => {
    try {
        dispatch({ type: CREATE_ORDER_REQUEST })
        const config = {
            headers: {
                'Content-Type': 'application/json'
            },
            withCredentials: true
        }
        const { data } = await axios.post(`${import.meta.env.VITE_API}/order/new`, order, config)
        dispatch({
            type: CREATE_ORDER_SUCCESS,
            payload: data
        })
        return data
```

backend/routes/order.js (lines 17-26)

```
router.post('/order/new', isAuthenticatedUser, newOrder);
router.get('/orders/me', isAuthenticatedUser, myOrders);
router.route('/order/:id').get(isAuthenticatedUser, getSingleOrder);
router.get('/admin/orders/', isAuthenticatedUser, authorizeRoles('admin'), allOrders);
// router.route('/admin/order/:id').delete(isAuthenticatedUser, deleteOrder);
router.route('/admin/order/:id').put(isAuthenticatedUser, updateOrder).delete(isAuthenticatedUser, deleteOrde
r);
router.get('/admin/total-orders', totalOrders);
router.get('/admin/total-sales', totalSales);
router.get('/admin/customer-sales', customerSales);
router.get('/admin/sales-per-month', salesPerMonth);
```

backend/controllers/order.js (lines 9-78)

```
exports.newOrder = async (req, res, next) => {
    const {
        orderItems,
        shippingInfo,
        itemsPrice,
        taxPrice,
        shippingPrice,
        totalPrice,
        paymentInfo

    } = req.body;

    const session = await mongoose.startSession()

    try {
        session.startTransaction()

        for (const item of orderItems) {
            const product = await Product.findOneAndUpdate(
                { _id: item.product, stock: { $gte: item.quantity }, isArchived: { $ne: true } },
                { $inc: { stock: -item.quantity } },
```

Feature Checklist Report

```
                { new: true, session }
            )

            if (!product) {
                const error = new Error(‘Insufficient stock or unavailable product: ${item.name}‘)
                error.statusCode = 400
                throw error
            }
        }

        const [order] = await Order.create([{
            orderItems,
            shippingInfo,
            itemsPrice,
            taxPrice,
            shippingPrice,
            totalPrice,
            paymentInfo,
            paidAt: Date.now(),
            user: req.user._id
        }], { session })

        await session.commitTransaction()

        const orderForEmail = await Order.findById(order._id).populate(‘user’, ’name email’)
        let emailSent = true
        try {
            await sendOrderEmail(
                orderForEmail,
                ’Your BrickNation order confirmation’,
                ’Thank you for your order!’
            )
        } catch (emailError) {
            emailSent = false
            console.error(‘Order confirmation email failed:’, emailError.message)
        }

        return res.status(200).json({
            success: true,
            order,
            emailSent
        })
    } catch (error) {
        if (session.inTransaction()) {
            await session.abortTransaction()
        }
        return next(error)
    } finally {
        await session.endSession()
```

backend/controllers/order.js (lines 137-171)

```
exports.updateOrder = async (req, res, next) => {
    const order = await Order.findById(req.params.id)
```

Feature Checklist Report

```
    if (!order) {
        return res.status(404).json({
            message: 'No Order found with this ID'
        })
    }

    if (order.orderStatus === 'Delivered') {
        return res.status(400).json({
            message: 'You have already delivered this order',
        })
    }

    order.orderStatus = req.body.status
    order.deliveredAt = Date.now()
    await order.save()

    let emailSent = true
    try {
        const orderForEmail = await Order.findById(order._id).populate('user', 'name email')
        await sendOrderEmail(
            orderForEmail,
            `Your BrickNation order status is now ${order.orderStatus}`,
            `Your order status has been updated to ${order.orderStatus}.`
        )
    } catch (emailError) {
        emailSent = false
        console.error('Order status email failed:', emailError.message)
    }

    return res.status(200).json({
        success: true,
        emailSent
    })
```

backend/utils/orderEmail.js (lines 1-91)

```
const PDFDocument = require('pdfkit')
const sendEmail = require('./sendEmail')

const money = value => `$${Number(value || 0).toFixed(2)}`

const escapeHtml = value => String(value || '')
    .replace(/&/g, '&amp;')
    .replace(/</g, '&lt;')
    .replace(/>/g, '&gt;')
    .replace(/"/g, '&quot;')
    .replace(/'/g, '&#039;')

const downloadImage = async url => {
    if (!url || !/^https?:\/\//i.test(url)) {
        return null
    }

    try {
```

Feature Checklist Report

```
        const response = await fetch(url, {
            signal: AbortSignal.timeout(10000)
        })
        if (!response.ok) {
            return null
        }
        return Buffer.from(await response.arrayBuffer())
    } catch (error) {
        console.error(‘Unable to download product image: ${url}‘, error.message)
        return null
    }
}

const prepareImages = async orderItems => {
    const images = await Promise.all(orderItems.map(async (item, index) => ({
        index,
        buffer: await downloadImage(item.image)
    })))

    return images.filter(image => image.buffer)
}

const createReceipt = async (order, productImages) => new Promise((resolve, reject) => {
    const document = new PDFDocument({ margin: 50 })
    const chunks = []

    document.on(’data’, chunk => chunks.push(chunk))
    document.on(’end’, () => resolve(Buffer.concat(chunks)))
    document.on(’error’, reject)

    document.fillColor(’#1f2937’).fontSize(22).text(’BrickNation’, { align: ’center’ })
    document.fontSize(16).text(’Official Order Receipt’, { align: ’center’ })
    document.moveDown()
    document.strokeColor(’#d1d5db’).moveTo(50, document.y).lineTo(545, document.y).stroke()
    document.moveDown()
    document.fillColor(’#374151’).fontSize(10)
    document.text(‘Order ID: ${order._id}‘)
    document.text(‘Order date: ${new Date(order.createdAt || Date.now()).toLocaleString()}‘)
    document.text(‘Order status: ${order.orderStatus}‘)
    document.moveDown()
    document.fillColor(’#111827’).fontSize(14).text(’Order details’)
    document.moveDown(0.5)

    order.orderItems.forEach((item, index) => {
        const image = productImages.find(productImage => productImage.index === index)
        const rowTop = document.y

        if (image) {
            document.image(image.buffer, 50, rowTop, { fit: [58, 58] })
        }

        document.fillColor(’#111827’).fontSize(10)
            .text(item.name, 120, rowTop, { width: 240 })
```

Feature Checklist Report

```
        document.fillColor('#4b5563').fontSize(9)
            .text(`${item.quantity} x ${money(item.price)}`, 120, rowTop + 17)
        document.fillColor('#111827').fontSize(10)
            .text(money(item.quantity * item.price), 430, rowTop + 8, { width: 100, align: 'right' })
        document.y = Math.max(rowTop + 68, document.y + 10)
    })

    document.moveDown()
    document.strokeColor('#d1d5db').moveTo(300, document.y).lineTo(545, document.y).stroke()
    document.moveDown(0.5)
    document.fillColor('#374151').fontSize(10)
        .text(`Subtotal: ${money(order.itemsPrice)}`, 330, document.y, { width: 215, align: 'right' })
        .text(`Shipping: ${money(order.shippingPrice)}`, 330, document.y, { width: 215, align: 'right' })
        .text(`Tax: ${money(order.taxPrice)}`, 330, document.y, { width: 215, align: 'right' })
    document.fillColor('#111827').fontSize(13)
        .text(`Grand total: ${money(order.totalPrice)}`, 300, document.y + 6, { width: 245, align: 'right' })
    document.end()
})

const sendOrderEmail = async (order, subject, intro) => {
```

backend/utils/orderEmail.js (lines 105-166)

```

        return `
            <tr>
                <td style="padding:14px 8px; border-bottom:1px solid #e5e7eb;">${imageMarkup}</td>
                <td style="padding:14px 8px; border-bottom:1px solid #e5e7eb; color:#111827;">${escapeHtml(item.name)}</td>
                <td style="padding:14px 8px; border-bottom:1px solid #e5e7eb; text-align:center; color:#4b5563;">${item.quantity}</td>
                <td style="padding:14px 8px; border-bottom:1px solid #e5e7eb; text-align:right; color:#4b5563;">${money(item.price)}</td>
                <td style="padding:14px 8px; border-bottom:1px solid #e5e7eb; text-align:right; font-weight:600; color:#111827;">${money(item.quantity * item.price)}</td>
            </tr>`
    }).join('')

    await sendEmail({
        email: order.user.email,
        subject,
        html: `
            <div style="margin:0; padding:32px 12px; background:#f3f4f6; font-family:Arial,Helvetica,sans-serif; color:#374151;">
                <div style="max-width:680px; margin:0 auto; background:#ffffff; border:1px solid #e5e7eb; border-radius:12px; overflow:hidden;">
                    <div style="padding:28px 32px; background:#1f2937; color:#ffffff;">
                        <div style="font-size:24px; font-weight:700; letter-spacing:.3px;">BrickNation</div>
                        <div style="margin-top:6px; font-size:13px; color:#d1d5db;">${escapeHtml(intro)}</div>
                    </div>
                    <div style="padding:28px 32px;">
                        <p style="margin:0 0 18px; font-size:15px;">Dear ${escapeHtml(order.user.name || 'Customer')},</p>
```

Feature Checklist Report

```
                        <p style="margin:0 0 22px; line-height:1.6;">Thank you for choosing BrickNation. Below are the details of your order.</p>
                        <div style="padding:14px 16px; background:#f9fafb; border-left:4px solid #2563eb; margin-bottom:24px;">
                            <div style="font-size:13px; color:#6b7280;">Order reference</div>
                            <div style="margin-top:4px; font-weight:700; color:#111827;">${escapeHtml(order._id)}</div>
                            <div style="margin-top:8px; font-size:13px; color:#6b7280;">Status: <strong style="color:#111827;">${escapeHtml(order.orderStatus)}</strong></div>
                        </div>
                        <table style="width:100%; border-collapse:collapse; font-size:13px;">
                            <thead>
                                <tr style="background:#f9fafb; color:#6b7280; text-align:left;">
                                    <th style="padding:10px 8px;">Image</th><th style="padding:10px 8px;">Product</th><th style="padding:10px 8px; text-align:center;">Qty</th><th style="padding:10px 8px; text-align:right;">Price</th><th style="padding:10px 8px; text-align:right;">Total</th>
                                </tr>
                            </thead>
                            <tbody>${itemsHtml}</tbody>
                        </table>
                        <div style="margin:24px 0 0 auto; max-width:280px; font-size:14px;">
                            <div style="display:flex; justify-content:space-between; padding:5px 0;">Subtotal <strong>${money(order.itemsPrice)}</strong></div>
                            <div style="display:flex; justify-content:space-between; padding:5px 0;">Shipping <strong>${money(order.shippingPrice)}</strong></div>
                            <div style="display:flex; justify-content:space-between; padding:5px 0;">Tax <strong>${money(order.taxPrice)}</strong></div>
                            <div style="display:flex; justify-content:space-between; border-top:2px solid #111827; margin-top:8px; padding:12px 0 0; font-size:17px; color:#111827;">Grand total <strong>${money(order.totalPrice)}</strong></div>
                        </div>
                        <p style="margin:28px 0 0; font-size:13px; line-height:1.6; color:#6b7280;">A formal PDF receipt is attached for your records. If you have any questions, please contact our support team.</p>
                        <p style="margin:24px 0 0;">Sincerely,<br><strong>The BrickNation Team</strong></p>
                    </div>
                    <div style="padding:18px 32px; background:#f9fafb; color:#9ca3af; font-size:11px; text-align:center;">This is an automated message. Please do not reply directly to this email.</div>
                </div>
            </div>
        ‘,
        attachments: [
            ...imageAttachments,
            {
                filename: ‘order-${order._id}.pdf‘,
                content: receipt,
                contentType: ’application/pdf’
            }
        ]
    })
}

module.exports = { sendOrderEmail }
```

Feature Checklist Report

==========

Unit 1 - Bad-word filtering and form validation

Review comments are cleaned with the bad-words package before saving and before being returned. React forms use Formik and Yup schemas for product, login, registration, shipping, payment, and user validation.

backend/utils/contentFilter.js (lines 1-7)

```
   1: const { Filter } = require('bad-words')
   2:
   3: const filter = new Filter()
   4:
   5: const filterComment = comment => filter.clean(String(comment || '').trim())
   6:
   7: module.exports = { filterComment }
```

backend/controllers/product.js (lines 319-325)

```
 319: exports.createProductReview = async (req, res, next) => {
 320:     const { rating, comment, productId } = req.body;
 321:     const numericRating = Number(rating)
 322:     const filteredComment = filterComment(comment)
 323:     if (!Number.isInteger(numericRating) || numericRating < 1 || numericRating > 5 || !filteredComment) {
 324:         return res.status(400).json({ success: false, message: 'Please provide a rating from 1 to 5 and a rev
      iew' })
 325:     }
```

backend/controllers/product.js (lines 394-408)

```
 394: exports.getProductReviews = async (req, res, next) => {
 395:     const reviews = await Review.find({ product_id: req.query.id })
 396:         .populate('user_id', 'name avatar')
 397:         .sort({ createdAt: -1 })
 398:     res.status(200).json({
 399:         success: true,
 400:         reviews: reviews.map(review => ({
 401:             ...review.toObject(),
 402:             comment: filterComment(review.comment),
 403:             user: review.user_id?._id,
 404:             name: review.user_id?.name,
 405:             avatar: review.user_id?.avatar
 406:         }))
 407:     })
 408: }
```

frontend/src/Components/Admin/NewProduct.jsx (lines 1-67)

```
   1: import { useState, useEffect } from 'react'
   2: import { useNavigate } from 'react-router-dom'
   3: import MetaData from '../Layout/MetaData'
   4: import Sidebar from './SideBar'
   5: // import { getToken } from '../../Utils/helpers';
   6: // import axios from 'axios'
   7: import { toast } from 'react-toastify';
   8: import 'react-toastify/dist/ReactToastify.css';
   9: import { useDispatch, useSelector } from 'react-redux'
  10: import { newProduct, clearErrors } from '../../actions/productActions'
```

Feature Checklist Report

```
import { NEW_PRODUCT_RESET } from '../../constants/productConstants'
import { useFormik } from 'formik'
import * as Yup from 'yup'
import {
    Box,
    Button,
    Container,
    FormControl,
    FormHelperText,
    InputLabel,
    MenuItem,
    Paper,
    Select,
    Stack,
    TextField,
    Typography
} from '@mui/material'

const NewProduct = () => {
    const dispatch = useDispatch()
    const { error, success } = useSelector(state => state.newProduct)
    const [images, setImages] = useState([]);
    const [imagesPreview, setImagesPreview] = useState([])
    const [imageError, setImageError] = useState('')
    const MAX_FILE_SIZE = 10 * 1024 * 1024
    // const [error, setError] = useState('')
    // const [loading, setLoading] = useState(true)
    // const [success, setSuccess] = useState('')
    // const [product, setProduct] = useState({})

    const categories = [
        'Materials',
        'Tools',
        'Equipment',
        'Design',
        'Consumables',
        'Electrical'
    ]

    let navigate = useNavigate()
    const formik = useFormik({
        initialValues: { name: '', price: 0, description: '', category: 'Materials', stock: 0, seller: '' },
        validationSchema: Yup.object({
            name: Yup.string().trim().required('Product name is required'),
            price: Yup.number().min(0, 'Price cannot be negative').required('Price is required'),
            description: Yup.string().trim().required('Description is required'),
            category: Yup.string().required('Category is required'),
            stock: Yup.number().integer().min(0, 'Stock cannot be negative').required('Stock is required'),
            seller: Yup.string().trim().required('Seller name is required')
        }),
        onSubmit: values => submitHandler(values)
    })

```

Feature Checklist Report

```
    const submitHandler = (values) => {
        if (imageError) {
            toast.error(imageError, { position: 'bottom-right' })
            return
```

frontend/src/Components/User/Login.jsx (lines 20-75)

```
import { useDispatch, useSelector } from 'react-redux'
import { useFormik } from 'formik'
import * as Yup from 'yup'
import { login, firebaseLogin, clearErrors } from '../../actions/userActions'
import {
    signInWithFacebook,
    signInWithGoogle,
} from '../../Utils/firebaseAuth'

const Login = () => {
    const { isAuthenticated, error, loading } = useSelector(state => state.auth)

    const [isLocked, setIsLocked] = useState(false)
    const [lockSecondsRemaining, setLockSecondsRemaining] = useState(0)

    const timerRef = useRef(null)
    const loginAttemptRef = useRef(false)
    const navigate = useNavigate()
    const location = useLocation()
    const dispatch = useDispatch()

    const startLockout = (seconds = 60) => {
        setIsLocked(true)
        setLockSecondsRemaining(seconds)
    }

    useEffect(() => {
        if (!isLocked) {
            return undefined
        }

        timerRef.current = setInterval(() => {
            setLockSecondsRemaining(current => {
                if (current <= 1) {
                    clearInterval(timerRef.current)
                    setIsLocked(false)
                    return 0
                }
                return current - 1
            })
        }, 1000)

        return () => {
            if (timerRef.current) {
                clearInterval(timerRef.current)
            }
        }
```

Feature Checklist Report

```
    }, [isLocked])

    const formik = useFormik({
        initialValues: { email: '', password: '' },
        validationSchema: Yup.object({
            email: Yup.string().email('Enter a valid email').required('Email is required'),
            password: Yup.string().required('Password is required')
        }),
        onSubmit: values => {
```

frontend/src/Components/User/Register.jsx (lines 18-43)

```
import { useFormik } from 'formik'
import * as Yup from 'yup'
import { toast } from 'react-toastify'
import 'react-toastify/dist/ReactToastify.css'

const Register = () => {
    const [avatar, setAvatar] = useState('')
    const [avatarPreview, setAvatarPreview] = useState('/images/default_avatar.png')
    const [avatarFileName, setAvatarFileName] = useState('Choose Avatar')
    const [avatarError, setAvatarError] = useState('')
    const [loading, setLoading] = useState(false)

    const navigate = useNavigate()
    const MAX_FILE_SIZE = 10 * 1024 * 1024 // 10MB in bytes

    const formik = useFormik({
        initialValues: { name: '', email: '', password: '' },
        validationSchema: Yup.object({
            name: Yup.string().trim().min(2, 'Name must be at least 2 characters').required('Name is required'),
            email: Yup.string().email('Enter a valid email').required('Email is required'),
            password: Yup.string().min(6, 'Password must be at least 6 characters').required('Password is required')
        }),
        onSubmit: values => {
            if (avatarError) {
                toast.error(avatarError, { position: 'bottom-right' })
                return
```

frontend/src/Components/Cart/Shipping.jsx (lines 7-30)

```
import { useFormik } from 'formik'
import * as Yup from 'yup'
import { Button, Container, FormControl, InputLabel, MenuItem, Paper, Select, Stack, TextField, Typography } from '@mui/material'

const Shipping = () => {
    const dispatch = useDispatch();
    const { shippingInfo } = useSelector(state => state.cart)
    const countriesList = Object.values(countries)
    const formik = useFormik({
        initialValues: {
            address: shippingInfo.address || '',
```

Feature Checklist Report

```
  18:             city: shippingInfo.city || ’’,
  19:             postalCode: shippingInfo.postalCode || ’’,
  20:             phoneNo: shippingInfo.phoneNo || ’’,
  21:             country: shippingInfo.country || ’’
  22:         },
  23:         validationSchema: Yup.object({
  24:             address: Yup.string().trim().required(’Address is required’),
  25:             city: Yup.string().trim().required(’City is required’),
  26:             postalCode: Yup.string().trim().required(’Postal code is required’),
  27:             phoneNo: Yup.string().matches(/^[0-9+\-\s()]{7,20}$/, ’Enter a valid phone number’).required(’Pho
      ne number is required’),
  28:             country: Yup.string().required(’Country is required’)
  29:         }),
  30:         onSubmit: values => {
```

==========

Unit 2 - User interface design with MUI

The selected 20-point implementation uses Material UI components such as Box, Paper, Button, Typography, DataGrid, Rating, Dialog, and Stack, combined with the existing CSS and Bootstrap styles.

frontend/src/Components/Product/ProductDetails.jsx (lines 3-8)

```
   3: import { Carousel } from ’react-bootstrap’
   4: import { Alert, Box, Button, Dialog, DialogActions, DialogContent, DialogTitle, Divider, IconButton, Paper, R
      ating, Stack, TextField, Typography } from ’@mui/material’
   5: import { useDispatch, useSelector } from ’react-redux’
   6: import MetaData from ’../Layout/MetaData’
   7: import ListReviews from ’../Review/ListReviews’
   8: import { getMyProductReview, getProductDetails, newReview } from ’../../actions/productActions’
```

frontend/src/Components/Product/ProductDetails.jsx (lines 55-75)

```
  55:                 <div className="col-12 col-lg-5 img-fluid" id="product_image">
  56:                     <Paper elevation={3} sx={{ p: 2, borderRadius: 2 }}>
  57:                         <Carousel pause="hover" className="product-carousel">
  58:                             {product.images && product.images.map(image => (
  59:                                 <Carousel.Item key={image.public_id}>
  60:                                     <img className="d-block w-100" src={image.url} alt={product.name} />
  61:                                 </Carousel.Item>
  62:                             ))}
  63:                         </Carousel>
  64:                     </Paper>
  65:                 </div>
  66:                 <div className="col-12 col-lg-5 mt-5">
  67:                     <Paper elevation={2} sx={{ p: 3, textAlign: ’left’, borderRadius: 2 }}>
  68:                         <Typography variant="h4" component="h1" sx={{ fontWeight: 600 }}>{product.name}</Typo
      graphy>
  69:                         <Divider sx={{ my: 2 }} />
  70:                         <Stack direction="row" alignItems="center" spacing={1.5}>
  71:                             <Rating value={product.ratings || 0} precision={0.5} readOnly size="small" />
  72:                             <Typography variant="body2" color="text.secondary">
  73:                                 {Number(product.ratings || 0).toFixed(1)} ({product.numOfReviews || 0} review
      s)
  74:                             </Typography>
  75:                         </Stack>
```

Feature Checklist Report

frontend/src/Components/Admin/ProductsList.jsx (lines 1-15)

```
   1: import { useEffect, useState } from 'react'
   2: import { Link } from 'react-router-dom'
   3: import { useDispatch, useSelector } from 'react-redux'
   4: import { formatPeso } from '../../Utils/helpers'
   5: import {
   6:     Alert,
   7:     Box,
   8:     Button,
   9:     Chip,
  10:     CircularProgress,
  11:     Dialog,
  12:     DialogActions,
  13:     DialogContent,
  14:     DialogTitle,
  15:     IconButton,
```

frontend/src/Components/Admin/OrdersList.jsx (lines 4-20)

```
   4: import { formatPeso } from '../../Utils/helpers'
   5: import {
   6:     Alert,
   7:     Box,
   8:     Button,
   9:     Chip,
  10:     CircularProgress,
  11:     Dialog,
  12:     DialogActions,
  13:     DialogContent,
  14:     DialogTitle,
  15:     IconButton,
  16:     Paper,
  17:     Stack,
  18:     Tooltip,
  19:     Typography
  20: } from '@mui/material'
```

frontend/src/Components/Admin/OrdersList.jsx (lines 182-226)

```
 182:                 <Box component="main" sx={{ flex: 1, minWidth: 0 }}>
 183:                     <Box sx={{ mb: 3 }}>
 184:                         <Typography variant="h4" fontWeight={700} sx={{ fontFamily: "'Space Grotesk', sans-se
     rif" }}>
 185:                             All Orders
 186:                         </Typography>
 187:                         <Typography color="text.secondary" variant="body2">
 188:                             Monitor customer purchases, shipping statuses, and order records
 189:                         </Typography>
 190:                     </Box>
 191:
 192:                     {error && <Alert severity="error" sx={{ mb: 2 }}>{error}</Alert>}
 193:
 194:                     <Paper elevation={0} sx={{ border: '1px solid', borderColor: 'divider', borderRadius: 3,
     p: 2 }}>
```

Feature Checklist Report

```
195:                         {loading ? (
196:                             <Stack alignItems="center" sx={{ py: 8 }}>
197:                                 <CircularProgress sx={{ color: 'var(--red, #e30620)' }} />
198:                                 <Typography color="text.secondary" sx={{ mt: 2 }}>Loading orders...</Typograp
    hy>
199:                             </Stack>
200:                         ) : (
201:                             <div style={{ width: '100%', minHeight: 450 }}>
202:                                 <DataGrid
203:                                     initialState={{
204:                                         sorting: {
205:                                             sortModel: [{ field: 'id', sort: 'desc' }],
206:                                         },
207:                                         pagination: {
208:                                             paginationModel: { pageSize: 10, page: 0 }
209:                                         }
210:                                     }}
211:                                     pageSizeOptions={[5, 10, 25, 50]}
212:                                     rows={rows}
213:                                     columns={columns}
214:                                     disableRowSelectionOnClick
215:                                     getRowId={(row) => row.id}
216:                                     sx={{
217:                                         border: 'none',
218:                                         '& .MuiDataGrid-cell': {
219:                                             display: 'flex',
220:                                             alignItems: 'center'
221:                                         },
222:                                         '& .MuiDataGrid-columnHeaders': {
223:                                             bgcolor: 'rgba(0, 0, 0, 0.02)',
224:                                             borderRadius: 2
225:                                         }
226:                                     }}
```

==========

Quiz 1 - Sales charts with date-range filters

The dashboard renders Recharts line and bar charts. Sales and products sold are calculated for a selectable date range, with Last 7 days and Last 30 days presets.

frontend/src/Components/Admin/SalesChart.jsx (lines 1-53)

```
  1: import { useMemo, useState } from 'react'
  2: import {
  3:     LineChart,
  4:     Line,
  5:     CartesianGrid,
  6:     XAxis,
  7:     YAxis,
  8:     Tooltip,
  9:     ResponsiveContainer
 10: } from 'recharts'
 11: import {
 12:     daysAgo,
 13:     eachDateKey,
```

Feature Checklist Report

```
    formatChartLabel,
    isOrderInRange,
    resolveDateRange,
    toInputDate
} from '../../Utils/chartDates'
import { formatPeso } from '../../Utils/helpers'

const buildDailySales = (orders, startDate, endDate) => {
    const { start, end } = resolveDateRange(startDate, endDate)
    const totals = new Map(eachDateKey(start, end).map(date => [date, 0]))

    orders.forEach(order => {
        if (!isOrderInRange(order, start, end)) {
            return
        }

        const key = toInputDate(order.paidAt || order.createdAt)
        if (!totals.has(key)) {
            return
        }

        totals.set(key, totals.get(key) + Number(order.totalPrice || 0))
    })

    return Array.from(totals.entries()).map(([date, total]) => ({
        date,
        total: Number(total.toFixed(2)),
        label: formatChartLabel(date)
    }))
}

const SalesChart = ({ orders = [] }) => {
    const [startDate, setStartDate] = useState(() => toInputDate(daysAgo(29)))
    const [endDate, setEndDate] = useState(() => toInputDate(new Date()))

    const sales = useMemo(
        () => buildDailySales(orders, startDate, endDate),
        [orders, startDate, endDate]
    )

```

frontend/src/Components/Admin/SalesChart.jsx (lines 88-145)

```
                <div className="sales-chart-header">
                    <div>
                        <h4 className="mb-1">Sales</h4>
                        <p className="text-muted mb-0">
                            Total in range: <b>?{formatPeso(rangeTotal)}</b>
                        </p>
                    </div>
                    <div className="sales-chart-filters">
                        <button type="button" className="btn btn-sm btn-outline-secondary" onClick={() => app
    lyPreset(7)}>
                            Last 7 days
```

Feature Checklist Report

```
                        </button>
                        <button type="button" className="btn btn-sm btn-outline-secondary" onClick={() => app
    lyPreset(30)}>
                            Last 30 days
                        </button>
                        <label className="mb-0">
                            From
                            <input
                                type="date"
                                className="form-control form-control-sm"
                                value={startDate}
                                max={endDate}
                                onChange={handleStartDateChange}
                            />
                        </label>
                        <label className="mb-0">
                            To
                            <input
                                type="date"
                                className="form-control form-control-sm"
                                value={endDate}
                                min={startDate}
                                max={toInputDate(new Date())}
                                onChange={handleEndDateChange}
                            />
                        </label>
                    </div>
                </div>

                {sales.length === 0 ? (
                    <p className="text-center text-muted mt-4 mb-0">No sales in this date range.</p>
                ) : (
                    <div className="sales-chart-wrap">
                        <ResponsiveContainer width="100%" height={360}>
                            <LineChart data={sales} margin={{ top: 16, right: 16, bottom: 8, left: 0 }}>
                                <CartesianGrid stroke="#eee7df" strokeDasharray="4 4" />
                                <XAxis dataKey="label" tick={{ fontSize: 12 }} interval="preserveStartEnd" />
                                <YAxis
                                    tick={{ fontSize: 12 }}
                                    tickFormatter={value => `?${Number(value).toLocaleString('en-PH', { maxim
    umFractionDigits: 0 })}`}
                                />
                                <Tooltip
                                    formatter={value => [`?${formatPeso(value)}`, 'Sales']}
                                    labelFormatter={(_, payload) => payload?.[0]?.payload?.date || ''}
                                />
                                <Line type="monotone" dataKey="total" stroke="#e30620" strokeWidth={2} dot={f
    alse} />
                            </LineChart>
                        </ResponsiveContainer>
                    </div>
```

frontend/src/Components/Admin/ProductsSoldChart.jsx (lines 1-55)

Feature Checklist Report

```
import { useMemo, useState } from 'react'
import {
    BarChart,
    Bar,
    CartesianGrid,
    XAxis,
    YAxis,
    Tooltip,
    ResponsiveContainer
} from 'recharts'
import {
    daysAgo,
    eachDateKey,
    formatChartLabel,
    isOrderInRange,
    resolveDateRange,
    toInputDate
} from '../../Utils/chartDates'

const orderQuantity = order =>
    (order.orderItems || []).reduce((sum, item) => sum + Number(item.quantity || 0), 0)

const buildDailyQuantities = (orders, startDate, endDate) => {
    const { start, end } = resolveDateRange(startDate, endDate)
    const totals = new Map(eachDateKey(start, end).map(date => [date, 0]))

    orders.forEach(order => {
        if (!isOrderInRange(order, start, end)) {
            return
        }

        const key = toInputDate(order.paidAt || order.createdAt)
        if (!totals.has(key)) {
            return
        }

        totals.set(key, totals.get(key) + orderQuantity(order))
    })

    return Array.from(totals.entries()).map(([date, sold]) => ({
        date,
        sold,
        label: formatChartLabel(date)
    }))
}

const ProductsSoldChart = ({ orders = [] }) => {
    const [startDate, setStartDate] = useState(() => toInputDate(daysAgo(29)))
    const [endDate, setEndDate] = useState(() => toInputDate(new Date()))

    const productsSold = useMemo(
        () => buildDailyQuantities(orders, startDate, endDate),
        [orders, startDate, endDate]
```

Feature Checklist Report

```
  54:     )
  55: 
```

frontend/src/Components/Admin/ProductsSoldChart.jsx (lines 90-146)

```
  90:                 <div className="sales-chart-header">
  91:                     <div>
  92:                         <h4 className="mb-1">Products sold</h4>
  93:                         <p className="text-muted mb-0">
  94:                             Total in range: <b>{rangeTotal}</b>
  95:                         </p>
  96:                     </div>
  97:                     <div className="sales-chart-filters">
  98:                         <button type="button" className="btn btn-sm btn-outline-secondary" onClick={() => app
      lyPreset(7)}>
  99:                             Last 7 days
 100:                         </button>
 101:                         <button type="button" className="btn btn-sm btn-outline-secondary" onClick={() => app
      lyPreset(30)}>
 102:                             Last 30 days
 103:                         </button>
 104:                         <label className="mb-0">
 105:                             From
 106:                             <input
 107:                                 type="date"
 108:                                 className="form-control form-control-sm"
 109:                                 value={startDate}
 110:                                 max={endDate}
 111:                                 onChange={handleStartDateChange}
 112:                             />
 113:                         </label>
 114:                         <label className="mb-0">
 115:                             To
 116:                             <input
 117:                                 type="date"
 118:                                 className="form-control form-control-sm"
 119:                                 value={endDate}
 120:                                 min={startDate}
 121:                                 max={toInputDate(new Date())}
 122:                                 onChange={handleEndDateChange}
 123:                             />
 124:                         </label>
 125:                     </div>
 126:                 </div>
 127: 
 128:                 {rangeTotal === 0 ? (
 129:                     <p className="text-center text-muted mt-4 mb-0">No products sold in this date range.</p>
 130:                 ) : (
 131:                     <div className="sales-chart-wrap">
 132:                         <ResponsiveContainer width="100%" height={360}>
 133:                             <BarChart data={productsSold} margin={{ top: 16, right: 16, bottom: 8, left: 0 }}
      >
 134:                                 <CartesianGrid stroke="#eee7df" strokeDasharray="4 4" />
 135:                                 <XAxis dataKey="label" tick={{ fontSize: 12 }} interval="preserveStartEnd" />
```

Feature Checklist Report

```
                                <YAxis allowDecimals={false} tick={{ fontSize: 12 }} />
                                <Tooltip
                                    formatter={value => [value, 'Products sold']}
                                    labelFormatter={(_, payload) => payload?.[0]?.payload?.date || ''}
                                />
                                <Bar dataKey="sold" fill="#c39a3b" radius={[4, 4, 0, 0]} />
                            </BarChart>
                        </ResponsiveContainer>
                    </div>
                )}
            </div>
```

frontend/src/Utils/chartDates.js (lines 1-76)

```
export const toInputDate = date => {
    const value = new Date(date)
    if (Number.isNaN(value.getTime())) {
        return ''
    }

    const year = value.getFullYear()
    const month = String(value.getMonth() + 1).padStart(2, '0')
    const day = String(value.getDate()).padStart(2, '0')
    return `${year}-${month}-${day}`
}

export const daysAgo = days => {
    const date = new Date()
    date.setDate(date.getDate() - days)
    return date
}

export const startOfDay = dateStr => {
    if (!dateStr) {
        return new Date(NaN)
    }

    const [year, month, day] = dateStr.split('-').map(Number)
    return new Date(year, month - 1, day, 0, 0, 0, 0)
}

export const endOfDay = dateStr => {
    if (!dateStr) {
        return new Date(NaN)
    }

    const [year, month, day] = dateStr.split('-').map(Number)
    return new Date(year, month - 1, day, 23, 59, 59, 999)
}

export const resolveDateRange = (startDate, endDate) => {
    let start = startOfDay(startDate)
    let end = endOfDay(endDate)

```

Feature Checklist Report

```
    if (Number.isNaN(start.getTime()) || Number.isNaN(end.getTime())) {
        return { start: new Date(NaN), end: new Date(NaN) }
    }

    if (start > end) {
        start = startOfDay(endDate)
        end = endOfDay(startDate)
    }

    return { start, end }
}

export const eachDateKey = (start, end) => {
    const keys = []
    const cursor = new Date(start)

    if (Number.isNaN(cursor.getTime()) || Number.isNaN(end.getTime())) {
        return keys
    }

    while (cursor <= end) {
        keys.push(toInputDate(cursor))
        cursor.setDate(cursor.getDate() + 1)
    }

    return keys
}

export const formatChartLabel = date =>
    new Date(‘${date}T00:00:00‘).toLocaleDateString(undefined, {
        month: ’short’,
        day: ’numeric’
    })

export const isOrderInRange = (order, start, end) => {
    const rawDate = order.paidAt || order.createdAt
```

frontend/src/Components/Admin/MonthlySalesChart.jsx (lines 1-45)

```
import React, { useState, useEffect } from ’react’
import { LineChart, Line, CartesianGrid, XAxis, YAxis, Tooltip, ResponsiveContainer } from ’recharts’;
import { getToken } from "../../Utils/helpers";
import axios from "axios";
import Loader from ’../Layout/Loader’;

export default function MonthlySalesChart() {
    const [sales, setSales] = useState([])
    const [loading, setLoading] = useState(true)
    const [error, setError] = useState(’’)
    const monthlySales = async () => {
        try {
            const config = {
                headers: {

```

Feature Checklist Report

```
                    'Authorization': 'Bearer ${getToken()}'
                }
            }

            const { data } = await axios.get('${import.meta.env.VITE_API}/admin/sales-per-month', config)
            setSales(data.salesPerMonth)
            // console.log(data.salesPerMonth)
            setLoading(false)

        } catch (error) {
            setError(error.response.data.message)
        }
    }
    useEffect(() => {
        monthlySales()

    }, [])

    return (
        <ResponsiveContainer width="90%" height={600}>
            <LineChart width={600} height={300} data={sales} margin={{ top: 5, right: 20, bottom: 5, left: 0
}}>
                <Line type="monotone" dataKey="total" stroke="#8884d8" />
                <CartesianGrid stroke="#ccc" strokeDasharray="5 5" />
                <XAxis dataKey="month" />
                <YAxis />
                <Tooltip />
            </LineChart>
        </ResponsiveContainer>
    );
}
```

backend/routes/order.js (lines 23-26)

```
router.get('/admin/total-orders', totalOrders);
router.get('/admin/total-sales', totalSales);
router.get('/admin/customer-sales', customerSales);
router.get('/admin/sales-per-month', salesPerMonth);
```

backend/controllers/order.js (lines 279-327)

```
exports.salesPerMonth = async (req, res, next) => {
    const salesPerMonth = await Order.aggregate([
        // Select
        // FROM
        // GROUP BY month(paidAt), year(paidAt)
        {
            $group: {
                // _id: {month: { $month: "$paidAt" } },
                _id: {
                    year: { $year: "$paidAt" },
                    month: { $month: "$paidAt" }
                },
                total: { $sum: "$totalPrice" },
            },
```

Feature Checklist Report

```
293:         },
294:
295:         {
296:             $addFields: {
297:                 month: {
298:                     $let: {
299:                         vars: {
300:                             monthsInString: [, 'Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sept'
     , 'Oct', 'Nov', 'Dec']
301:                         },
302:                         in: {
303:                             $arrayElemAt: ['$$monthsInString', "$_id.month"]
304:                         }
305:                     }
306:                 }
307:             }
308:         },
309:         { $sort: { "_id.month": 1 } },
310:         {
311:             $project: {
312:                 _id: false,
313:                 month: true,
314:                 total: true,
315:             }
316:         }
317:
318:     ])
319:     if (!salesPerMonth) {
320:         return res.status(404).json({
321:             message: 'error sales per month',
322:         })
323:     }
324:     // return console.log(customerSales)
325:     res.status(200).json({
326:         success: true,
327:         salesPerMonth
```

==========

Quiz 2 - Infinite scroll on the homepage

The homepage observes a sentinel element with IntersectionObserver. When it enters the viewport and more products exist, the page number increments; the action appends the next page instead of replacing existing products.

frontend/src/Components/Home.jsx (lines 90-94)

```
 90:     const dispatch = useDispatch();
 91:     const { loading, products, filteredProductsCount } = useSelector(state => state.products);
 92:     const loadMoreRef = useRef(null)
 93:     // console.log(products)
 94:     // const getProducts = async (keyword = '', price, page = 1) => {
```

frontend/src/Components/Home.jsx (lines 169-185)

```
169:         }
170:     }, [queryCategory, categories])
```

Feature Checklist Report

```
171:
172:     useEffect(() => {
173:         dispatch(getProducts(currentPage, keyword, price, category, rating, reviews, currentPage > 1))
174:     }, [currentPage, keyword, price, category, rating, reviews, dispatch]);
175:
176:     useEffect(() => {
177:         const loadMore = entries => {
178:             if (entries[0].isIntersecting && !loading && products.length < filteredProductsCount) {
179:                 setCurrentPage(page => page + 1)
180:             }
181:         }
182:         const observer = new IntersectionObserver(loadMore, { rootMargin: '300px' })
183:         if (loadMoreRef.current) observer.observe(loadMoreRef.current)
184:         return () => observer.disconnect()
185:     }, [filteredProductsCount, loading, products.length])
```

frontend/src/Components/Home.jsx (lines 398-404)

```
398:                                 )}
399:
400:                             </div>
401:                             {products.length < filteredProductsCount && (
402:                                 <div ref={loadMoreRef} className="d-flex justify-content-center mt-5" style={
    { minHeight: 48 }}>
403:                                     {loading && <CircularProgress size={32} />}
404:                                 </div>
```

frontend/src/actions/productActions.jsx (lines 45-80)

```
 45: export const getProducts = (
 46:     currentPage = 1,
 47:     keyword = '',
 48:     price = [0, 100000],
 49:     category = '',
 50:     rating = '',
 51:     reviews = '',
 52:     append = false
 53: ) => async (dispatch) => {
 54:     try {
 55:         dispatch({
 56:             type: ALL_PRODUCTS_REQUEST,
 57:             append
 58:         })
 59:
 60:         const params = new URLSearchParams({
 61:             page: currentPage.toString(),
 62:             limit: '8',
 63:             keyword,
 64:             'price[lte]': price[1].toString(),
 65:             'price[gte]': price[0].toString()
 66:         })
 67:         if (Array.isArray(category)) {
 68:             category.forEach(categoryId => params.append('category', categoryId))
 69:         } else if (category) {
 70:             params.set('category', category)
```

Feature Checklist Report

```
        }
        if (rating) params.set('rating[gte]', rating.toString())
        if (reviews) params.set('numOfReviews[gte]', reviews.toString())
        const link = `${import.meta.env.VITE_API}/products?${params.toString()}`
        const { data } = await axios.get(link)
        console.log(data)
        dispatch({
            type: ALL_PRODUCTS_SUCCESS,
            payload: data,
            meta: { append }
```

frontend/src/reducers/productReducers.jsx (lines 43-69)

```
    GET_REVIEWS_FAIL,
    DELETE_REVIEW_REQUEST,
    DELETE_REVIEW_SUCCESS,
    DELETE_REVIEW_FAIL,
    CLEAR_ERRORS
} from '../constants/productConstants'

export const productsReducer = (state = { products: [] }, action) => {
    switch (action.type) {
        case ADMIN_PRODUCTS_REQUEST:
        case ALL_PRODUCTS_REQUEST:
            return {
                ...state,
                loading: true,
                products: action.append ? state.products : []
            }


        case ALL_PRODUCTS_SUCCESS:
            return {
                ...state,
                loading: false,
                products: action.meta?.append
                    ? [...state.products, ...action.payload.products.filter(product =>
                        !state.products.some(existingProduct => existingProduct._id === product._id)
                    )]
                    : action.payload.products,
```

backend/utils/apiFeatures.js (lines 80-90)

```
        return this;
    }

    pagination(resPerPage) {
        const currentPage = Number(this.queryStr.page) || 1;
        const skip = resPerPage * (currentPage - 1);

        this.query = this.query.limit(resPerPage).skip(skip);
        return this;
    }
}
```

Feature Checklist Report

==========

Quiz 3 - Google/Facebook login with Firebase

Firebase handles the Google or Facebook popup in the browser. The resulting Firebase ID token is sent to the backend, where Firebase Admin verifies it and the backend creates a normal application session cookie.

frontend/src/firebase.js (lines 1-15)

```
   1: import { initializeApp } from 'firebase/app'
   2: import { getAuth } from 'firebase/auth'
   3:
   4: const firebaseConfig = {
   5:     apiKey: import.meta.env.VITE_FIREBASE_API_KEY,
   6:     authDomain: import.meta.env.VITE_FIREBASE_AUTH_DOMAIN,
   7:     projectId: import.meta.env.VITE_FIREBASE_PROJECT_ID,
   8:     storageBucket: import.meta.env.VITE_FIREBASE_STORAGE_BUCKET,
   9:     messagingSenderId: import.meta.env.VITE_FIREBASE_MESSAGING_SENDER_ID,
  10:     appId: import.meta.env.VITE_FIREBASE_APP_ID,
  11: }
  12:
  13: const app = initializeApp(firebaseConfig)
  14:
  15: export const firebaseAuth = getAuth(app)
```

frontend/src/Utils/firebaseAuth.jsx (lines 1-20)

```
   1: import {
   2:     FacebookAuthProvider,
   3:     GoogleAuthProvider,
   4:     signInWithPopup,
   5: } from 'firebase/auth'
   6:
   7: import { firebaseAuth } from '../firebase'
   8:
   9: const googleProvider = new GoogleAuthProvider()
  10: const facebookProvider = new FacebookAuthProvider()
  11:
  12: googleProvider.setCustomParameters({
  13:     prompt: 'select_account',
  14: })
  15:
  16: export const signInWithGoogle = () =>
  17:     signInWithPopup(firebaseAuth, googleProvider)
  18:
  19: export const signInWithFacebook = () =>
  20:     signInWithPopup(firebaseAuth, facebookProvider)
```

frontend/src/Components/User/Login.jsx (lines 91-103)

```
  91:             })
  92:             return
  93:         }
  94:
  95:         try {
  96:             const result = await providerLogin()
  97:             const idToken = await result.user.getIdToken()
```

Feature Checklist Report

```
  98:             dispatch(firebaseLogin(idToken))
  99:         } catch (error) {
 100:             const isPopupDismissed = [
 101:                 'auth/popup-closed-by-user',
 102:                 'auth/cancelled-popup-request'
 103:             ].includes(error.code)
```

frontend/src/actions/userActions.jsx (lines 164-184)

```
 164: export const firebaseLogin = (idToken) => async (dispatch) => {
 165:     try {
 166:         dispatch({ type: LOGIN_REQUEST })
 167:
 168:         const { data } = await axios.post(
 169:             `${import.meta.env.VITE_API}/firebase-login`,
 170:             { idToken },
 171:             { withCredentials: true }
 172:         )
 173:
 174:         dispatch({
 175:             type: LOGIN_SUCCESS,
 176:             payload: data.user
 177:         })
 178:     } catch (error) {
 179:         dispatch({
 180:             type: LOGIN_FAIL,
 181:             payload: error.response?.data?.message ||
 182:                 error.message ||
 183:                 'Social login failed'
 184:         })
```

backend/routes/auth.js (lines 20-28)

```
  20:     logout
  21: } = require('../controllers/auth');
  22: const { isAuthenticatedUser, authorizeRoles } = require('../middlewares/auth')
  23:
  24:
  25: router.post('/register', upload.single("avatar"), registerUser);
  26: router.post('/login', loginUser);
  27: router.post('/firebase-login', firebaseLogin);
  28: router.post('/password/forgot', forgotPassword);
```

backend/controllers/auth.js (lines 67-117)

```
  67: exports.firebaseLogin = async (req, res) => {
  68:     const { idToken } = req.body
  69:
  70:     if (!idToken) {
  71:         return res.status(400).json({
  72:             success: false,
  73:             message: 'Firebase ID token is required'
  74:         })
  75:     }
  76:
  77:     try {
```

Feature Checklist Report

```
        const firebaseAdmin = require('../config/firebaseAdmin')
        const decodedToken = await firebaseAdmin.auth.verifyIdToken(idToken)
        const email = decodedToken.email

        if (!email) {
            return res.status(400).json({
                success: false,
                message: 'This social account does not provide an email address'
            })
        }

        const provider = decodedToken.firebase?.sign_in_provider
        if (!['google.com', 'facebook.com'].includes(provider)) {
            return res.status(400).json({
                success: false,
                message: 'Only Google and Facebook accounts are supported'
            })
        }

        let user = await User.findOne({ email })
        const profile = {
            name: decodedToken.name || email.split('@')[0],
            firebaseUid: decodedToken.uid,
            authProvider: provider,
            ...(decodedToken.picture && {
                avatar: {
                    public_id: 'avatars/default',
                    url: decodedToken.picture
                }
            })
        }

        if (!user) {
            user = await User.create({
                email,
                ...profile
            })
        } else {
            if (user.isDeleted) {
                return res.status(403).json({
```

backend/config/firebaseAdmin.js (lines 1-30)

```
const { getApps, initializeApp, cert } = require('firebase-admin/app')
const { getAuth } = require('firebase-admin/auth')

const {
    FIREBASE_PROJECT_ID,
    FIREBASE_CLIENT_EMAIL,
    FIREBASE_PRIVATE_KEY,
} = process.env

if (!FIREBASE_PROJECT_ID ||
    !FIREBASE_CLIENT_EMAIL ||
```

Feature Checklist Report

```
    !FIREBASE_PRIVATE_KEY) {
    throw new Error(
        'Missing FIREBASE_PROJECT_ID, FIREBASE_CLIENT_EMAIL, or FIREBASE_PRIVATE_KEY'
    )
}

const firebaseApp = getApps().length
    ? getApps()[0]
    : initializeApp({
        credential: cert({
            projectId: FIREBASE_PROJECT_ID,
            clientEmail: FIREBASE_CLIENT_EMAIL,
            privateKey: FIREBASE_PRIVATE_KEY.replace(/\\n/g, '\n'),
        }),
    })

module.exports = {
    auth: getAuth(firebaseApp),
}
```

==========

Quiz 4 - Redux state management

Redux combines feature reducers into one store, uses thunk middleware for asynchronous Axios actions, and exposes the store to React through the Provider.

frontend/src/store.js (lines 1-71)

```
import { legacy_createStore as createStore, combineReducers, applyMiddleware } from 'redux';
import { thunk } from 'redux-thunk';

import {
    productsReducer,
    productDetailsReducer,
    newProductReducer,
    productReducer,
    newReviewReducer,
    adminReviewsReducer,
} from './reducers/productReducers'
import {
    authReducer,
    userReducer,
    allUsersReducer,
    forgotPasswordReducer,
    // allUsersReducer,
    // userDetailsReducer,
} from './reducers/userReducers'

import { cartReducer } from './reducers/cartReducers';

import { newOrderReducer,
    myOrdersReducer,
    orderDetailsReducer,
    allOrdersReducer,
    orderReducer,
```

Feature Checklist Report

```
28:  } from ’./reducers/orderReducers’;
29:
30: let initialState = {
31:     cart: {
32:         cartItems: localStorage.getItem(’cartItems’)
33:             ? JSON.parse(localStorage.getItem(’cartItems’))
34:             : [],
35:         shippingInfo: localStorage.getItem(’shippingInfo’)
36:             ? JSON.parse(localStorage.getItem(’shippingInfo’))
37:             : {}
38:     },
39:     user: {},
40:     auth: {
41:         user: null,
42:         loading: true,
43:         isAuthenticated: false
44:     },
45:
46: }
47:
48: const reducer = combineReducers({
49:     products: productsReducer,
50:     productDetails: productDetailsReducer,
51:     auth: authReducer,
52:     user: userReducer,
53:     allUsers: allUsersReducer,
54:     forgotPassword: forgotPasswordReducer,
55:     newProduct: newProductReducer,
56:     cart: cartReducer,
57:     newOrder: newOrderReducer,
58:     myOrders: myOrdersReducer,
59:     orderDetails: orderDetailsReducer,
60:     product: productReducer,
61:     newReview: newReviewReducer,
62:     adminReviews: adminReviewsReducer,
63:     allOrders: allOrdersReducer,
64:     order: orderReducer,
65:     // allUsers: allUsersReducer,
66:     // userDetails: userDetailsReducer,
67: })
68:
69: const middlware = [thunk]
70: const store = createStore(reducer, initialState, applyMiddleware(...middlware))
71: export default store;
```

frontend/src/main.jsx (lines 1-12)

```
1: import { StrictMode } from ’react’
2: import { createRoot } from ’react-dom/client’
3: import ’./index.css’
4: import App from ’./App.jsx’
5: import { Provider } from ’react-redux’
6: import store from ’./store’
7:
```

Feature Checklist Report

```
createRoot(document.getElementById('root')).render(
    <Provider store={store}>
        <App />
    </Provider>
)
```

frontend/src/actions/productActions.jsx (lines 45-82)

```
export const getProducts = (
    currentPage = 1,
    keyword = '',
    price = [0, 100000],
    category = '',
    rating = '',
    reviews = '',
    append = false
) => async (dispatch) => {
    try {
        dispatch({
            type: ALL_PRODUCTS_REQUEST,
            append
        })

        const params = new URLSearchParams({
            page: currentPage.toString(),
            limit: '8',
            keyword,
            'price[lte]': price[1].toString(),
            'price[gte]': price[0].toString()
        })
        if (Array.isArray(category)) {
            category.forEach(categoryId => params.append('category', categoryId))
        } else if (category) {
            params.set('category', category)
        }
        if (rating) params.set('rating[gte]', rating.toString())
        if (reviews) params.set('numOfReviews[gte]', reviews.toString())
        const link = `${import.meta.env.VITE_API}/products?${params.toString()}`
        const { data } = await axios.get(link)
        console.log(data)
        dispatch({
            type: ALL_PRODUCTS_SUCCESS,
            payload: data,
            meta: { append }
        })

```

frontend/src/actions/orderActions.jsx (lines 1-45)

```
import axios from 'axios'
import {
    CREATE_ORDER_REQUEST,
    CREATE_ORDER_SUCCESS,
    CREATE_ORDER_FAIL,
    MY_ORDERS_REQUEST,
```

Feature Checklist Report

```
    MY_ORDERS_SUCCESS,
    MY_ORDERS_FAIL,
    ORDER_DETAILS_REQUEST,

    ORDER_DETAILS_SUCCESS,

    ORDER_DETAILS_FAIL,
    ALL_ORDERS_REQUEST,

    ALL_ORDERS_SUCCESS,

    ALL_ORDERS_FAIL,
    UPDATE_ORDER_SUCCESS,

    UPDATE_ORDER_REQUEST,

    UPDATE_ORDER_FAIL,
    DELETE_ORDER_REQUEST,
    DELETE_ORDER_SUCCESS,
    DELETE_ORDER_FAIL,
    CLEAR_ERRORS

} from '../constants/orderConstants'

export const createOrder = (order) => async (dispatch) => {
    try {
        dispatch({ type: CREATE_ORDER_REQUEST })
        const config = {
            headers: {
                'Content-Type': 'application/json'
            },
            withCredentials: true
        }
        const { data } = await axios.post(`${import.meta.env.VITE_API}/order/new`, order, config)
        dispatch({
            type: CREATE_ORDER_SUCCESS,
            payload: data
        })
        return data
```